

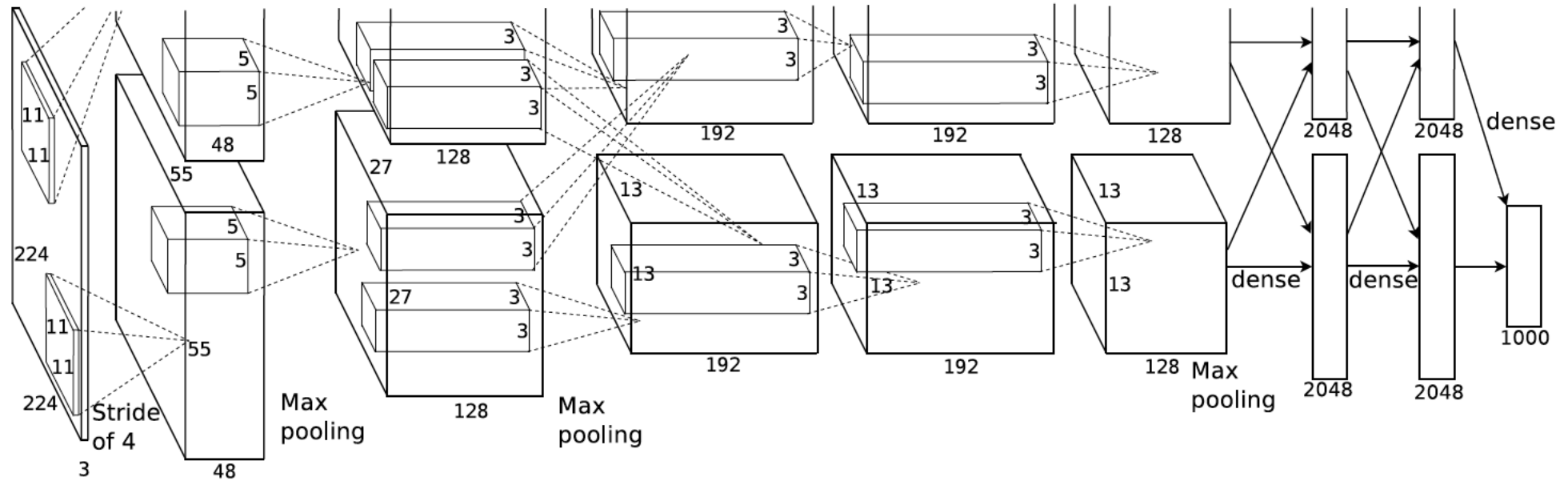
Neural Networks

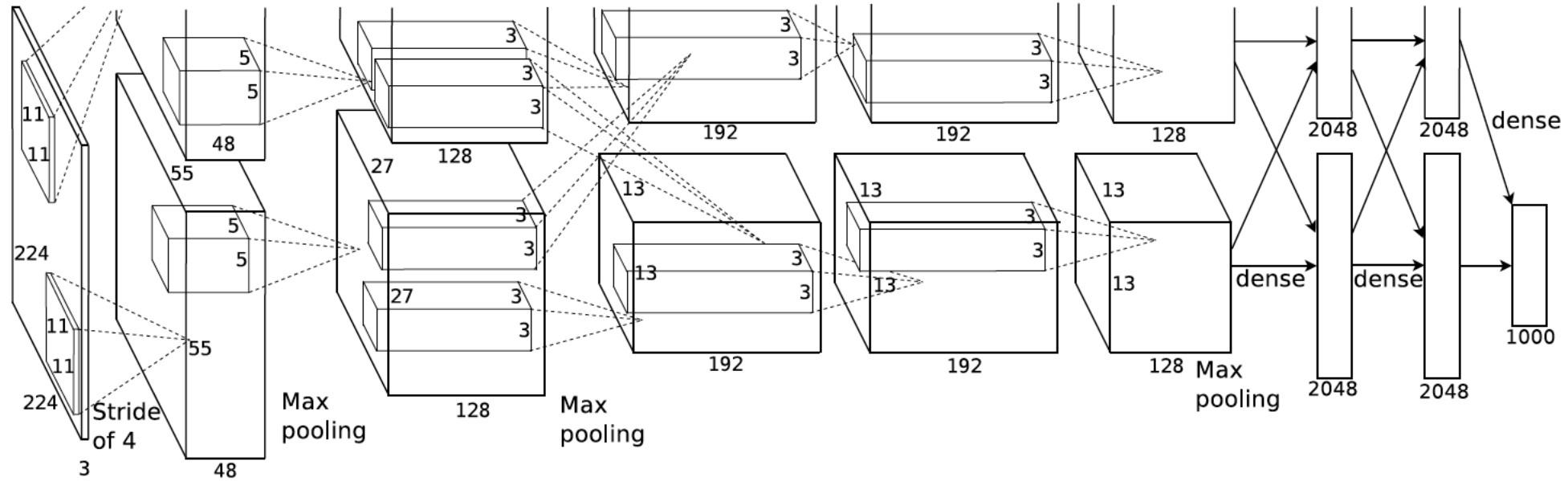
Yu Kong

Department of Computer Science and Engineering

Michigan State University

AlexNet





```
# AlexNet model
class AlexNet(Sequential):
    def __init__(self, input_shape, num_classes):
        super().__init__()

        self.add(Conv2D(96, kernel_size=(11,11), strides= 4,
            padding= 'valid', activation= 'relu',
            input_shape= input_shape,
            kernel_initializer= 'he_normal'))
        self.add(MaxPooling2D(pool_size=(3,3), strides= (2,2),
            padding= 'valid', data_format= None))

        self.add(Conv2D(256, kernel_size=(5,5), strides= 1,
            padding= 'same', activation= 'relu',
            kernel_initializer= 'he_normal'))
        self.add(MaxPooling2D(pool_size=(3,3), strides= (2,2),
            padding= 'valid', data_format= None))

        self.add(Conv2D(384, kernel_size=(3,3), strides= 1,
            padding= 'same', activation= 'relu',
            kernel_initializer= 'he_normal'))
```

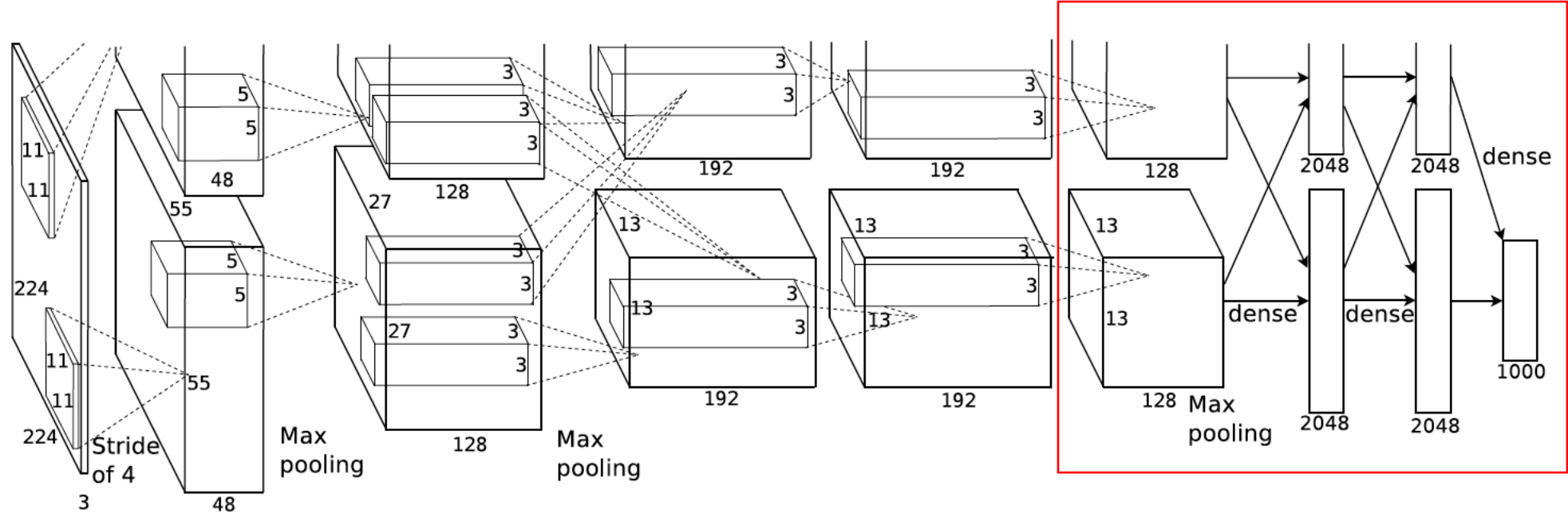
```
        self.add(Conv2D(384, kernel_size=(3,3), strides= 1,
            padding= 'same', activation= 'relu',
            kernel_initializer= 'he_normal'))

        self.add(Conv2D(256, kernel_size=(3,3), strides= 1,
            padding= 'same', activation= 'relu',
            kernel_initializer= 'he_normal'))

        self.add(MaxPooling2D(pool_size=(3,3), strides= (2,2),
            padding= 'valid', data_format= None))

        self.add(Flatten())
        self.add(Dense(4096, activation= 'relu'))
        self.add(Dense(4096, activation= 'relu'))
        self.add(Dense(1000, activation= 'relu'))
        self.add(Dense(num_classes, activation= 'softmax'))

        self.compile(optimizer= tf.keras.optimizers.Adam(0.001),
            loss= 'categorical_crossentropy',
            metrics= ['accuracy'])
```



1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Image -> Conv ->

->MaxPooling(2,2) ->?-> Flatten->?-> w1=(0.5,1,0.5,1)->z?->a?->w2=(0.1,0.2)->logit?

CNN Training Procedure

A typical training procedure for a neural network is as follows:

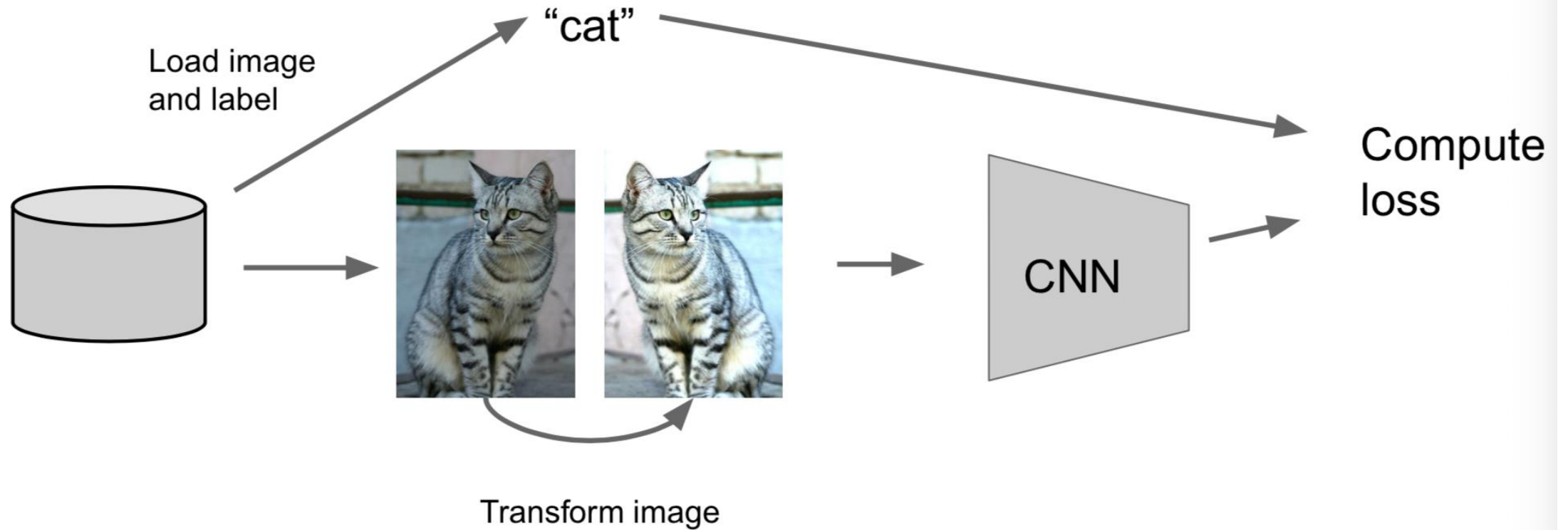
- Define the neural network that has some learnable parameters (or weights)
- Iterate over a dataset of inputs
- Process input through the network
- Compute the loss (how far is the output from being correct)
- Propagate gradients back into the network's parameters
- Update the weights of the network, typically using a simple update rule: $\text{weight} = \text{weight} - \text{learning_rate} * \text{gradient}$

```
learning_rate = 0.01
for f in net.parameters():
    f.data.sub_(f.grad.data * learning_rate)
```

CNN Preprocessing Pipeline

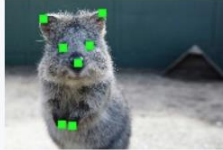
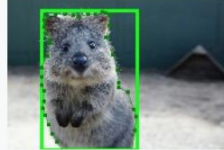
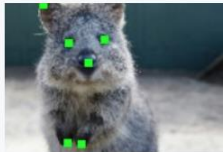
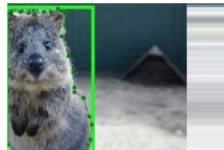
1. Randomly sample an image and decode it into 32-bit floating point raw pixel values in $[0, 255]$.
2. Randomly crop a rectangular region whose aspect ratio is randomly sampled in $[3/4, 4/3]$ and area randomly sampled in $[8\%, 100\%]$, then resize the cropped region into a 224-by-224 square image (**RandomResizedCrop**).
3. Flip horizontally with 0.5 probability (**RandomHorizontalFlip**).
4. Scale hue, saturation, and brightness with coefficients uniformly drawn from $[0.6, 1.4]$.
5. Add PCA noise with a coefficient sampled from a normal distribution $N(0, 0.1)$.
6. Normalize RGB channels by subtracting 123.68, 116.779, 103.939 and dividing by 58.393, 57.12, 57.375, respectively (**Normalize**).

Data Augmentation



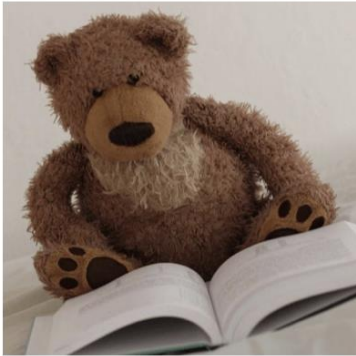
Data augmentation can be performed either in real-time or off-line.

Data Augmentation

	Image	Heatmaps	Seg. Maps	Keypoints	Bounding Boxes, Polygons
<i>Original Input</i>					
Gauss. Noise + Contrast + Sharpen					
Affine					
Crop + Pad					
Fliplr + Perspective					

Less prone to overfitting

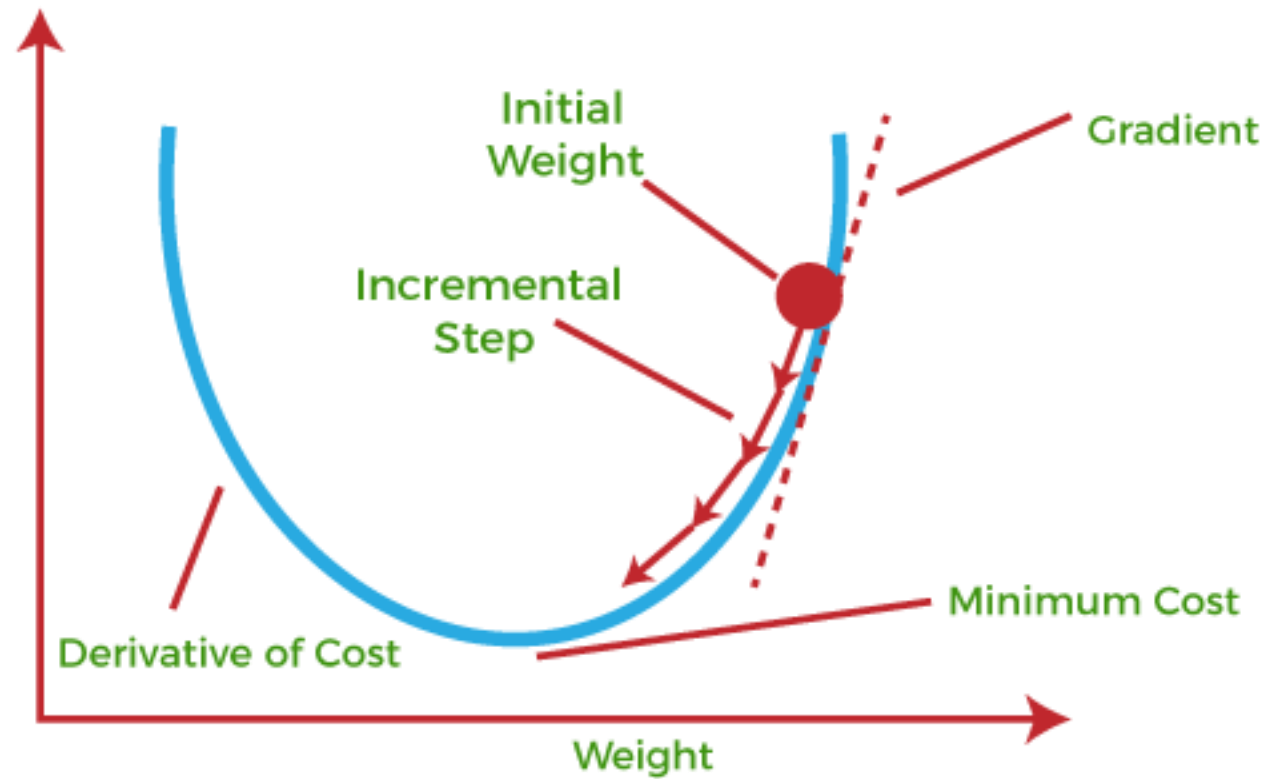
Data Augmentation

Original	Flip	Rotation	Random crop
			
Color shift	Noise addition	Information loss	Contrast change
			

Data Augmentation Implementation

```
# Data augmentation and normalization for training  
# Just normalization for validation  
data_transforms = {  
    'train': transforms.Compose([  
        transforms.RandomResizedCrop(224),  
        transforms.RandomHorizontalFlip(),  
        transforms.ToTensor(),  
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])  
    ]),  
    'val': transforms.Compose([  
        transforms.Resize(256),  
        transforms.CenterCrop(224),  
        transforms.ToTensor(),  
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])  
    ]),  
}
```

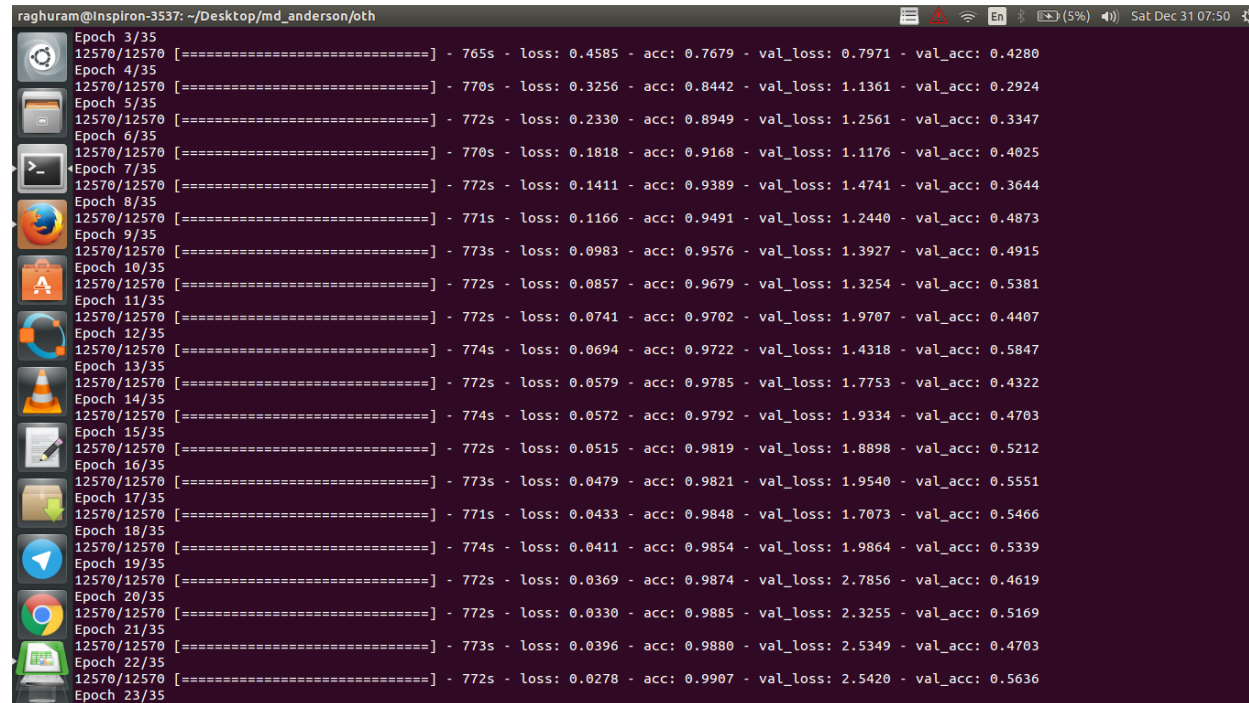
Epoch, Batch, Mini batch



$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial E}{\partial \mathbf{w}}$$

Epoch, Batch, Mini batch

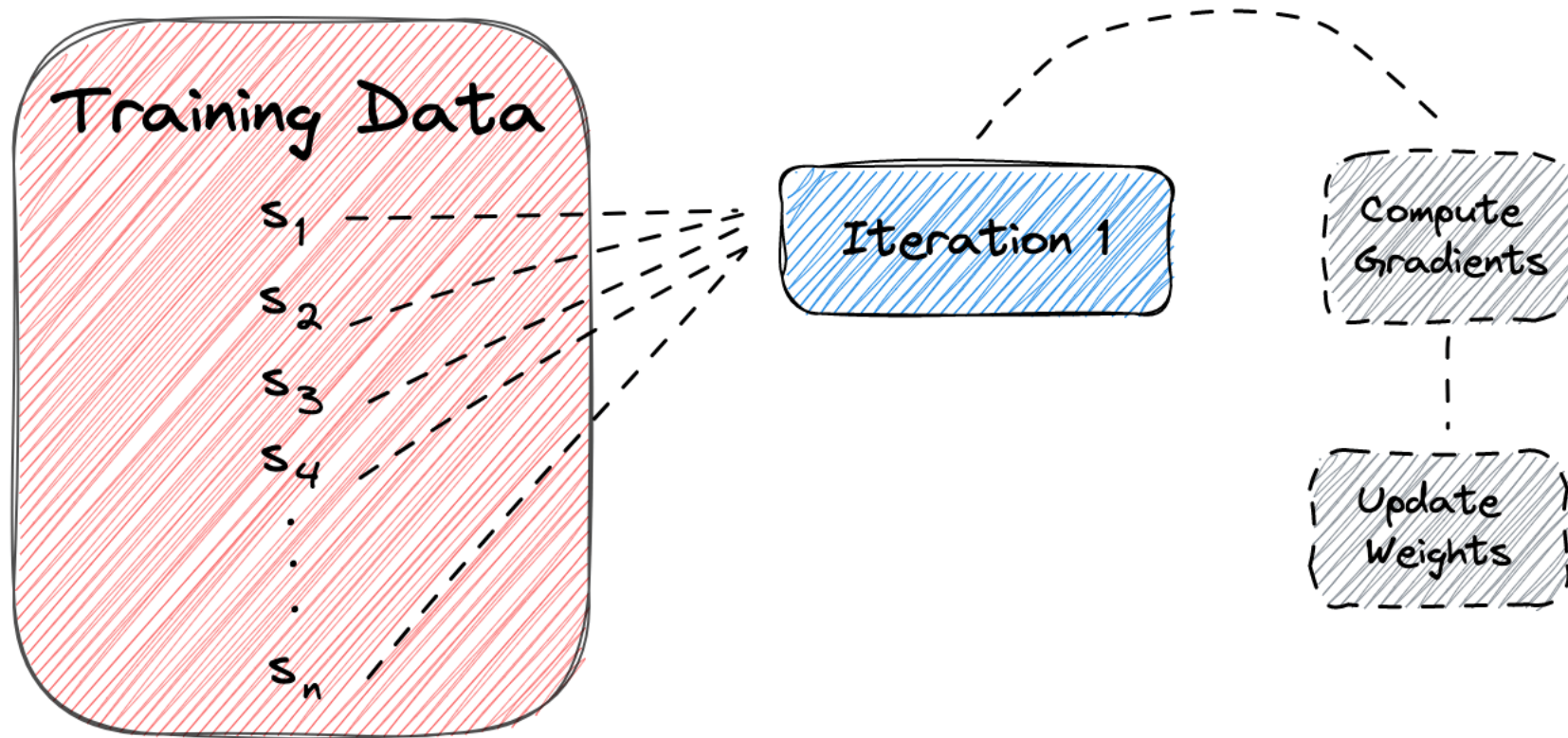
- Epoch: we have passed each sample of the training set one time through the network to update the parameters



A terminal window titled 'raghuram@Inspiron-3537: ~/Desktop/md_anderson/oth' displays the output of a training process. The output shows 23 epochs, each with a progress bar, time taken, loss, accuracy, validation loss, and validation accuracy. The progress bars are composed of equals signs. The training metrics show a general downward trend in loss and an upward trend in accuracy over the epochs.

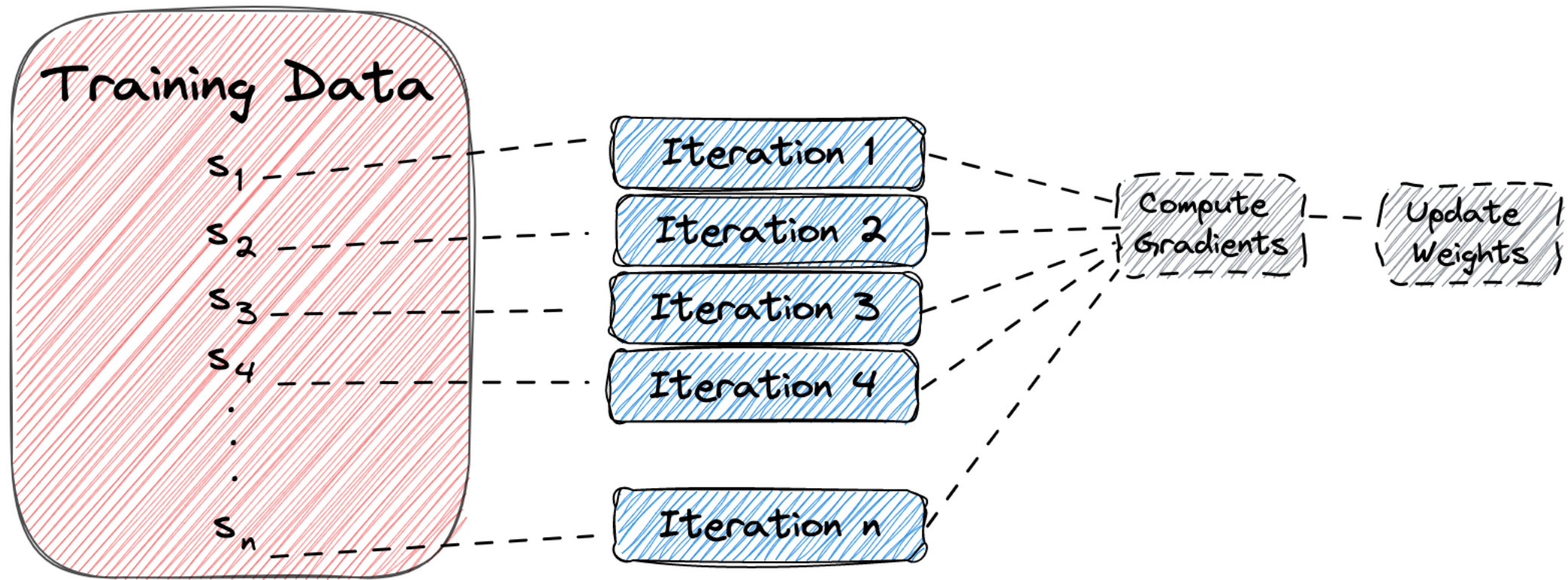
Epoch	Time	loss	acc	val_loss	val_acc
3/35	765s	0.4585	0.7679	0.7971	0.4280
4/35	770s	0.3256	0.8442	1.1361	0.2924
5/35	772s	0.2330	0.8949	1.2561	0.3347
6/35	770s	0.1818	0.9168	1.1176	0.4025
7/35	772s	0.1411	0.9389	1.4741	0.3644
8/35	771s	0.1166	0.9491	1.2440	0.4873
9/35	773s	0.0983	0.9576	1.3927	0.4915
10/35	772s	0.0857	0.9679	1.3254	0.5381
11/35	772s	0.0741	0.9702	1.9707	0.4407
12/35	774s	0.0694	0.9722	1.4318	0.5847
13/35	772s	0.0579	0.9785	1.7753	0.4322
14/35	774s	0.0572	0.9792	1.9334	0.4703
15/35	772s	0.0515	0.9819	1.8898	0.5212
16/35	773s	0.0479	0.9821	1.9540	0.5551
17/35	771s	0.0433	0.9848	1.7073	0.5466
18/35	774s	0.0411	0.9854	1.9864	0.5339
19/35	772s	0.0369	0.9874	2.7856	0.4619
20/35	772s	0.0330	0.9885	2.3255	0.5169
21/35	773s	0.0396	0.9880	2.5349	0.4703
22/35	772s	0.0278	0.9907	2.5420	0.5636
23/35					

Batch Gradient Descent



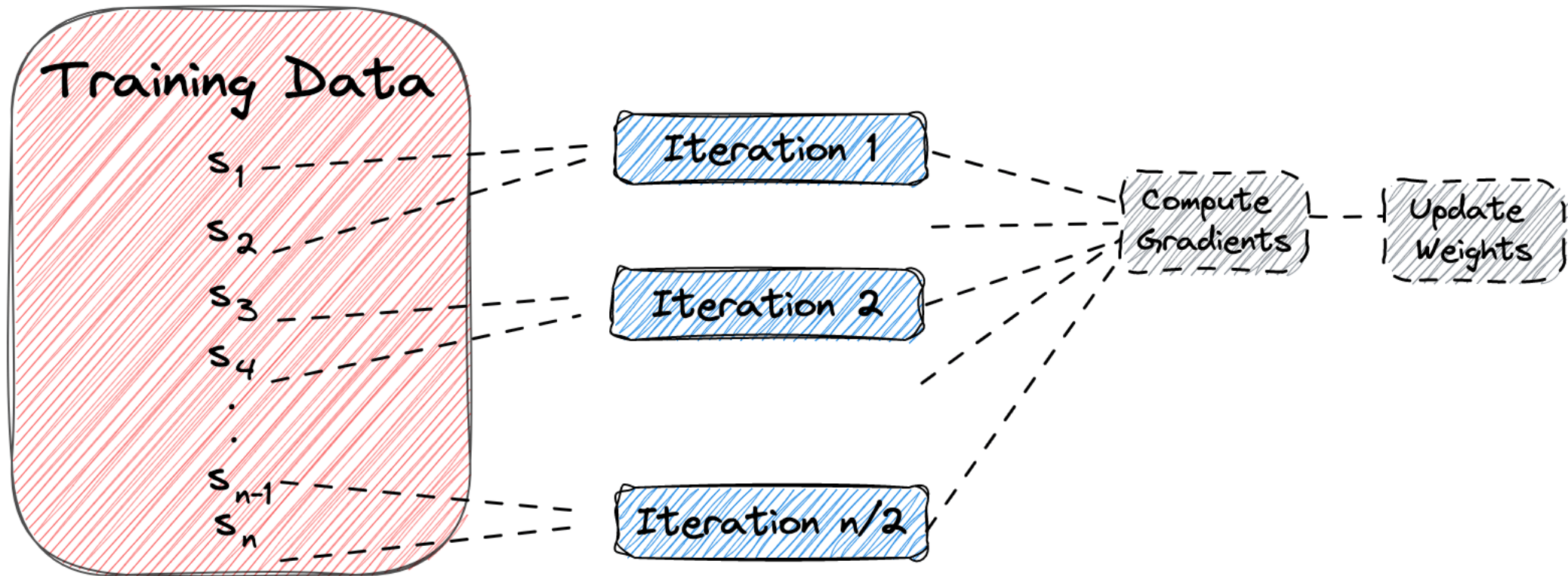
All training data are used in a single iteration of the gradient descent.

Stochastic Gradient Descent



Only one sample is used in a single iteration of the gradient descent.

Mini-Batch Gradient Descent



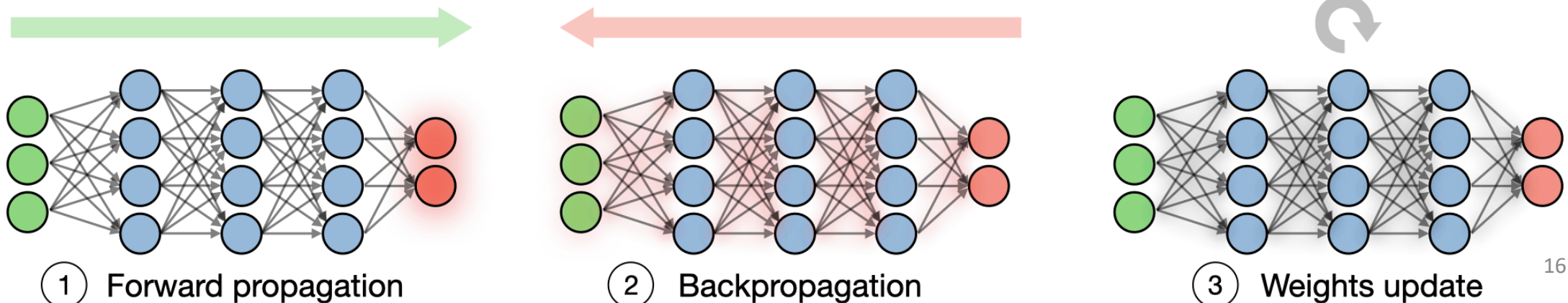
A group of samples (mini-batch) are used in a single iteration of the gradient descent.

Training

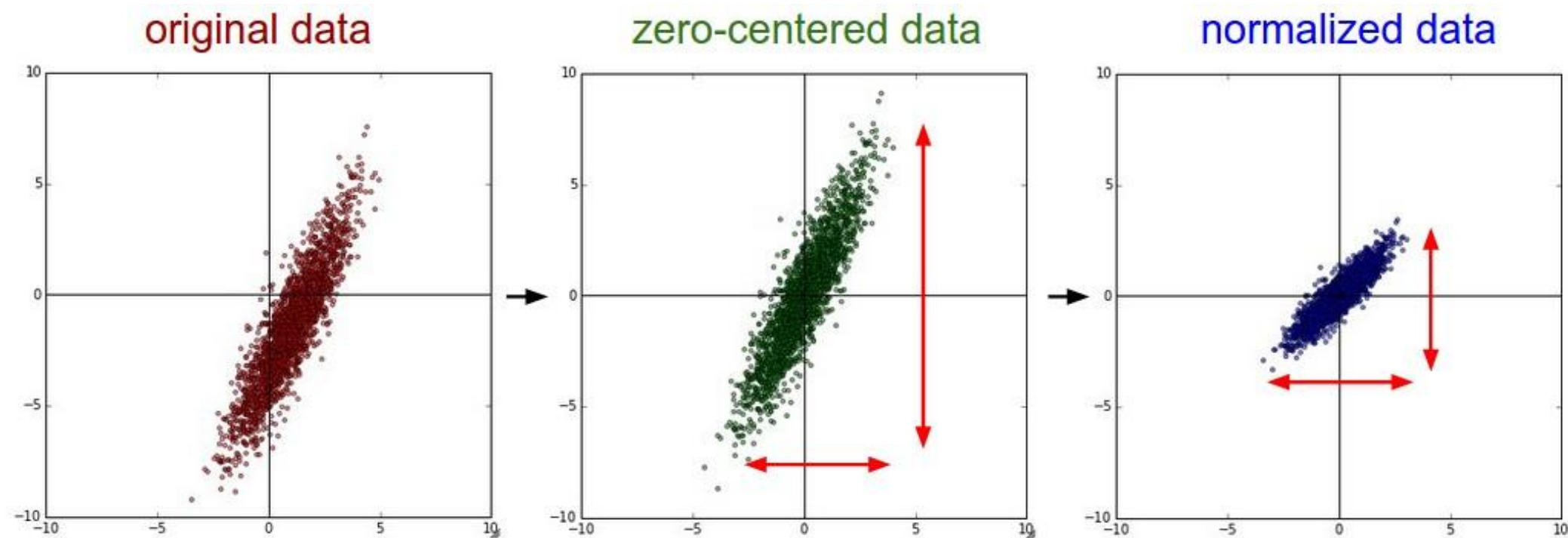
Algorithm 1 Train a neural network with mini-batch stochastic gradient descent.

```
initialize(net)
for epoch = 1, ..., K do
  for batch = 1, ..., #images/b do
    images  $\leftarrow$  uniformly random sample  $b$  images
     $X, y \leftarrow$  preprocess(images)
     $z \leftarrow$  forward(net,  $X$ )
     $\ell \leftarrow$  loss( $z, y$ )
    grad  $\leftarrow$  backward( $\ell$ )
    update(net, grad)
  end for
end for
```

$$w \leftarrow w - \alpha \frac{\partial L(z, y)}{\partial w}$$



Data Preprocessing



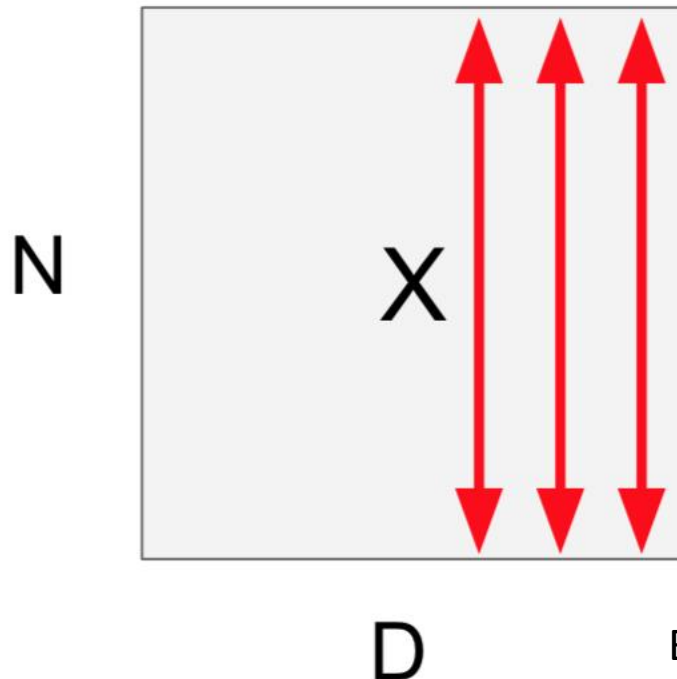
[0,255]: Normalize RGB channels by subtracting 123.68, 116.779, 103.939 and dividing by 58.393, 57.12, 57.375, respectively

[0,1]: Normalize RGB channels by subtracting 0.485, 0.456, 0.406 and dividing by 0.229, 0.224, 0.225, respectively

Batch Normalizations

zero-mean unit-variance activations

Input: $x : N \times D$



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

Per-channel mean,
shape is D

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

Per-channel var,
shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

Normalized x,
Shape is N x D

Batch Normalization for fully-connected networks

Batch Normalizations

Input: $x : N \times D$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

Learning $\gamma = \sigma$,
 $\beta = \mu$ will recover the
identity function!

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \text{Per-channel mean, shape is } D$$

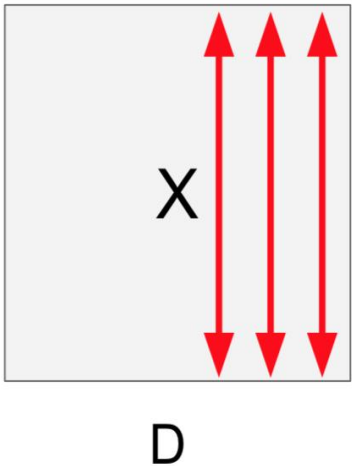
$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad \text{Per-channel var, shape is } D$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}} \quad \text{Normalized x, Shape is } N \times D$$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j \quad \text{Output, Shape is } N \times D$$

Batch Normalizations

Input: $x : N \times D$



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

Per-channel mean,
shape is D

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

Per-channel var,
shape is D

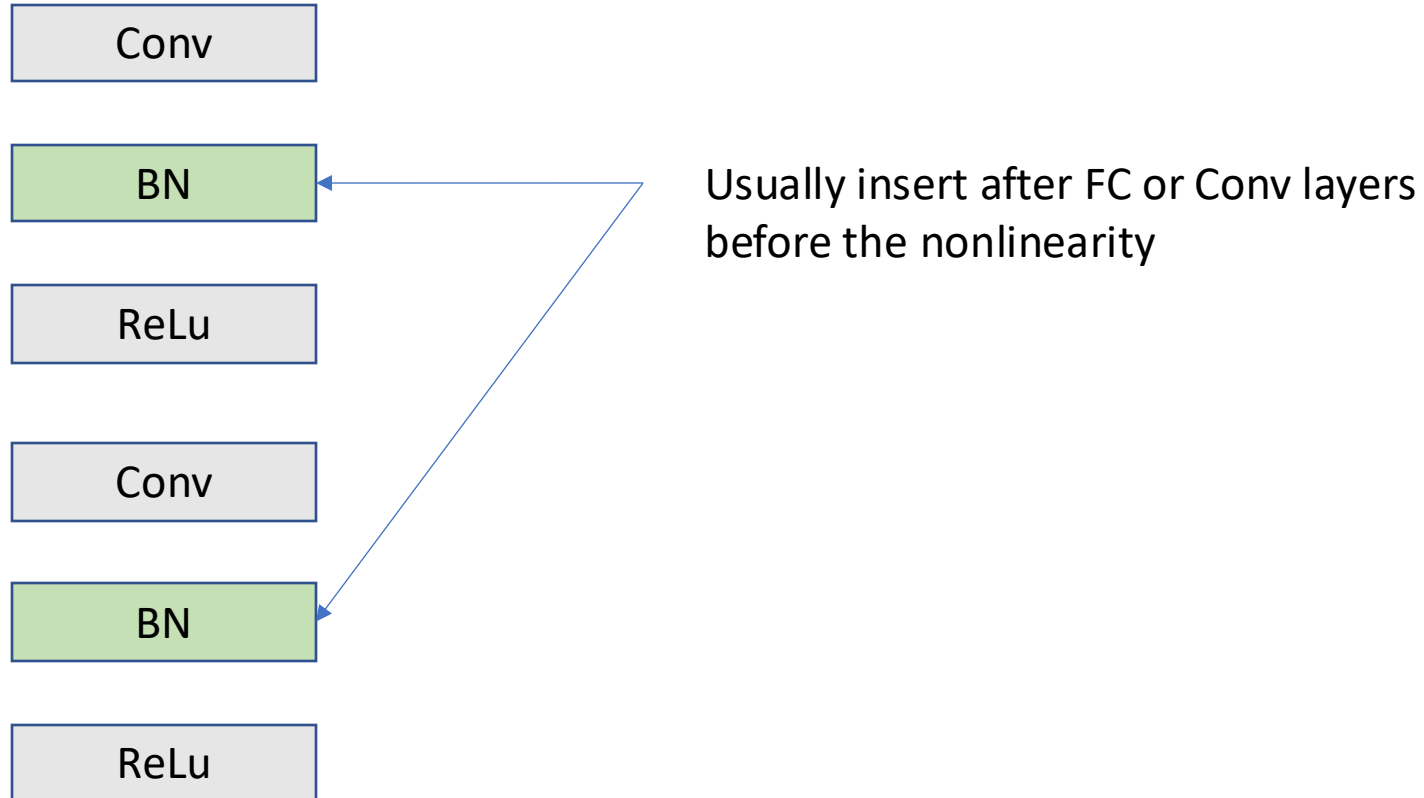
$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

Normalized x,
Shape is N x D

```
X = torch.randn(20,100) * 5 + 10
X = Variable(X)
```

```
mu = torch.mean(X[:,1])
var_ = torch.var(X[:,1], unbiased=False)
sigma = torch.sqrt(var_ + 1e-5)
x = (X[:,1] - mu)/sigma
```

Batch Normalizations



```
out = self.conv1(x)
out = self.bn1(out)
out = self.relu(out)
```

```
out = self.conv2(out)
out = self.bn2(out)
out = self.relu(out)
```

```
out = self.conv3(out)
out = self.bn3(out)
```

Resnet bottleneck

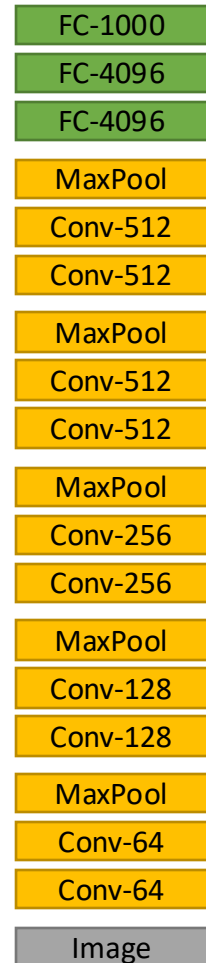
How to Train CNNs?

Two major transfer learning scenarios:

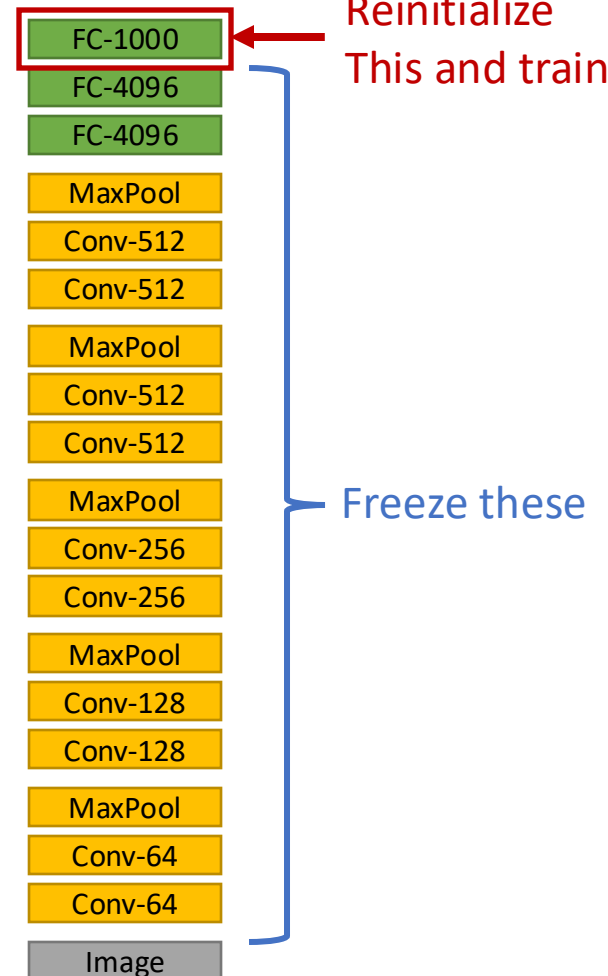
- **Finetuning the convnet:** Instead of random initialization, we initialize the network with a pretrained network, like the one that is trained on imagenet 1000 dataset. Rest of the training looks as usual.
- **ConvNet as fixed feature extractor:** Here, we will freeze the weights for all of the network except that of the final fully connected layer. This last fully connected layer is replaced with a new one with random weights and only this layer is trained.

How to Train CNNs?

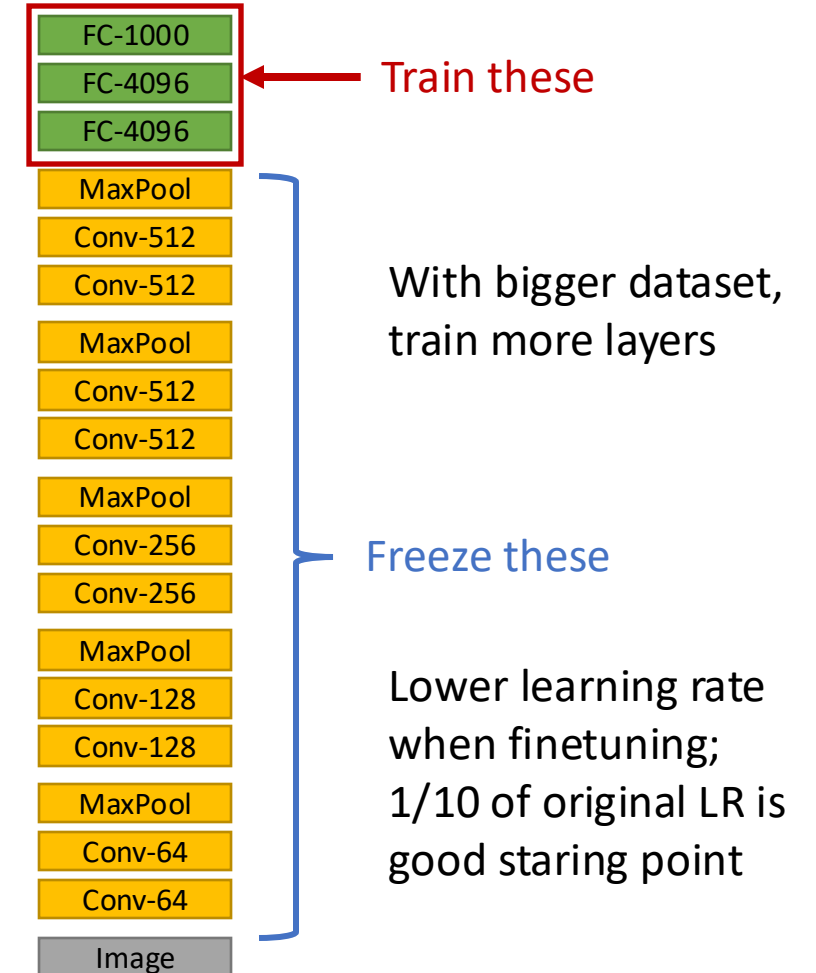
1. Train on ImageNet



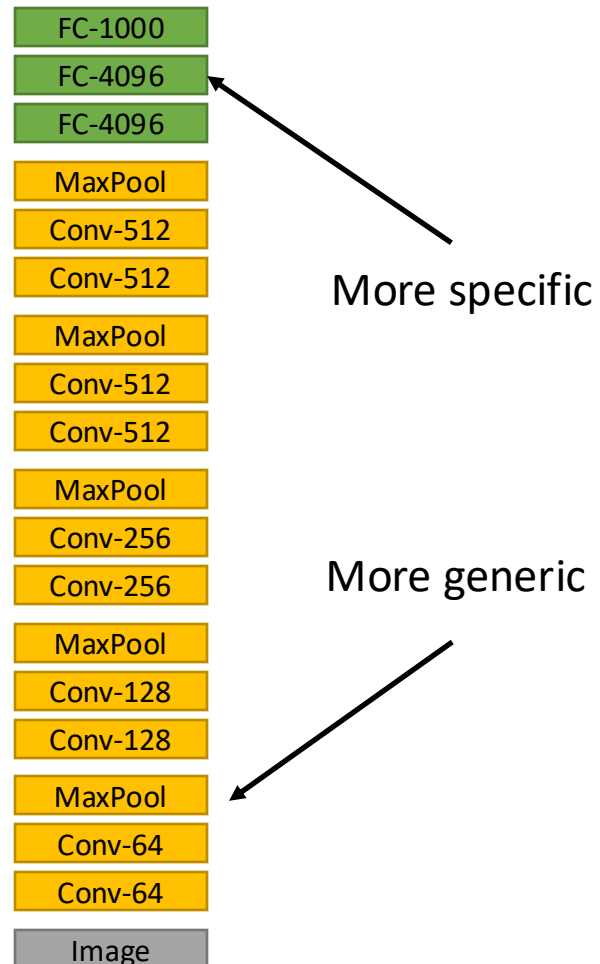
2. Small Dataset (C classes)



3. Bigger dataset



How to Train CNNs?



	Very similar dataset	Very different dataset
Very little data	Use linear classifier on top layer	Not good! Try linear classifier
Quite a lot of data	Finetune a few layers	Finetune a large number of layers

How to Train CNNs?

```
import torchvision.models as models
resnet18 = models.resnet18(pretrained=True)
alexnet = models.alexnet(pretrained=True)
squeezenet = models.squeezenet1_0(pretrained=True)
vgg16 = models.vgg16(pretrained=True)
densenet = models.densenet161(pretrained=True)
inception = models.inception_v3(pretrained=True)
googlenet = models.googlenet(pretrained=True)
shufflenet = models.shufflenet_v2_x1_0(pretrained=True)
mobilenet = models.mobilenet_v2(pretrained=True)
resnext50_32x4d = models.resnext50_32x4d(pretrained=True)
wide_resnet50_2 = models.wide_resnet50_2(pretrained=True)
mnasnet = models.mnasnet1_0(pretrained=True)
```

TORCHVISION.MODELS

How to Train CNNs?

```
model_ft = models.resnet18(pretrained=True)
```

Model
definition

```
num_fts = model_ft.fc.in_features
```

```
# Here the size of each output sample is set to 2.
```

```
# Alternatively, it can be generalized to nn.Linear(num_fts, len(class_names)).
```

```
model_ft.fc = nn.Linear(num_fts, 2)
```

Model
finetuned

```
model_ft = model_ft.to(device)
```

```
criterion = nn.CrossEntropyLoss()
```

Loss function

```
# Observe that all parameters are being optimized
```

```
optimizer_ft = optim.SGD(model_ft.parameters(), lr=0.001, momentum=0.9)
```

Optimizer

```
# Decay LR by a factor of 0.1 every 7 epochs
```

```
exp_lr_scheduler = lr_scheduler.StepLR(optimizer_ft, step_size=7, gamma=0.1)
```

Learning rate
policy

```
model_ft = train_model(model_ft, criterion, optimizer_ft, exp_lr_scheduler,  
                        num_epochs=25)
```

Model Training

How to Train CNNs?

```
model_conv = torchvision.models.resnet18(pretrained=True)
for param in model_conv.parameters():
    param.requires_grad = False  # freeze the parameters

# Parameters of newly constructed modules have requires_grad=True by default
num_ftrs = model_conv.fc.in_features
model_conv.fc = nn.Linear(num_ftrs, 2)

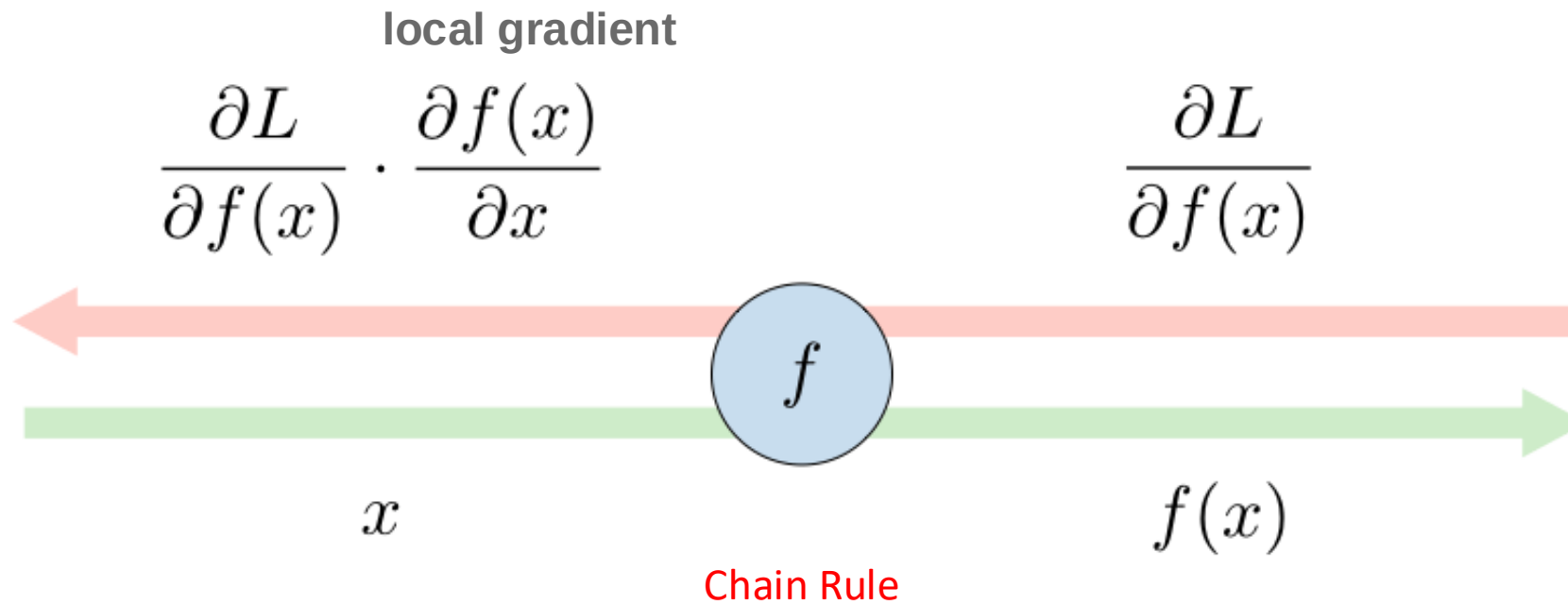
model_conv = model_conv.to(device)

criterion = nn.CrossEntropyLoss()

# Observe that only parameters of final layer are being optimized as
# opposed to before.
optimizer_conv = optim.SGD(model_conv.fc.parameters(), lr=0.001, momentum=0.9)

# Decay LR by a factor of 0.1 every 7 epochs
exp_lr_scheduler = lr_scheduler.StepLR(optimizer_conv, step_size=7, gamma=0.1)
```

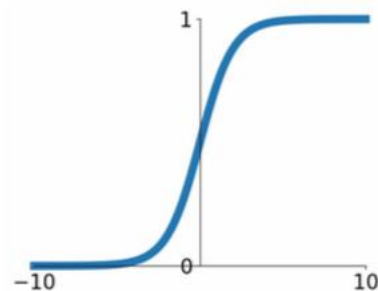
Backpropagation



Activation Functions

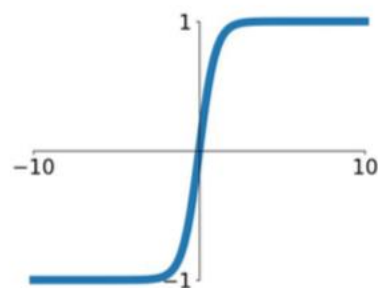
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



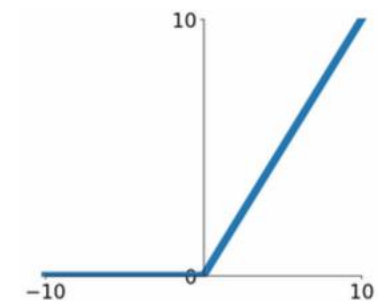
tanh

$$\tanh(x)$$



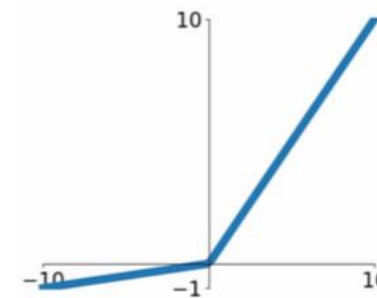
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

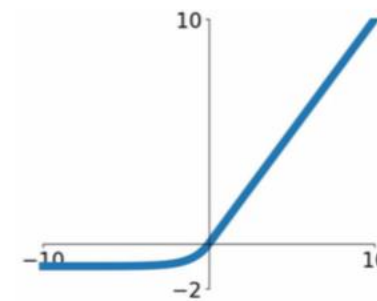


Maxout

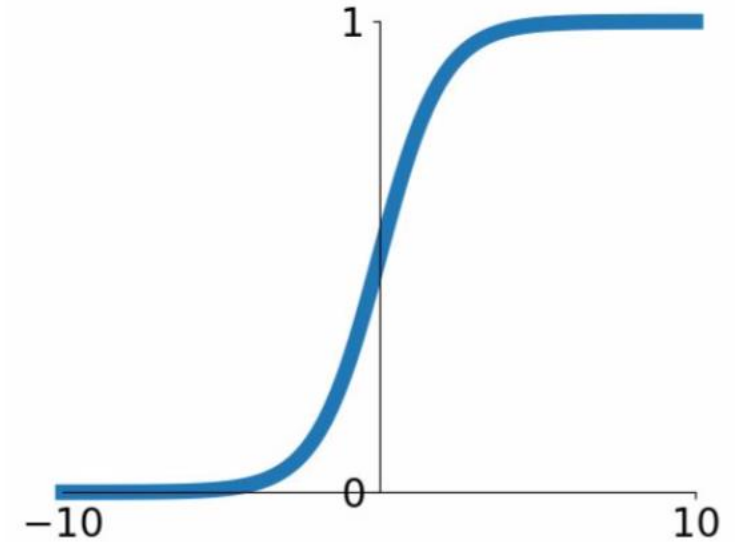
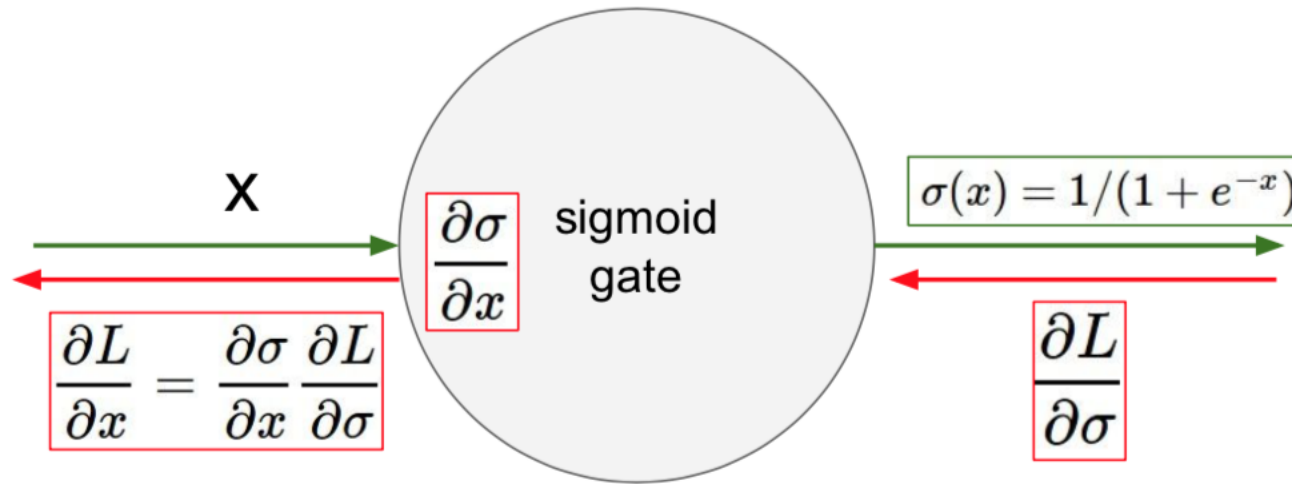
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



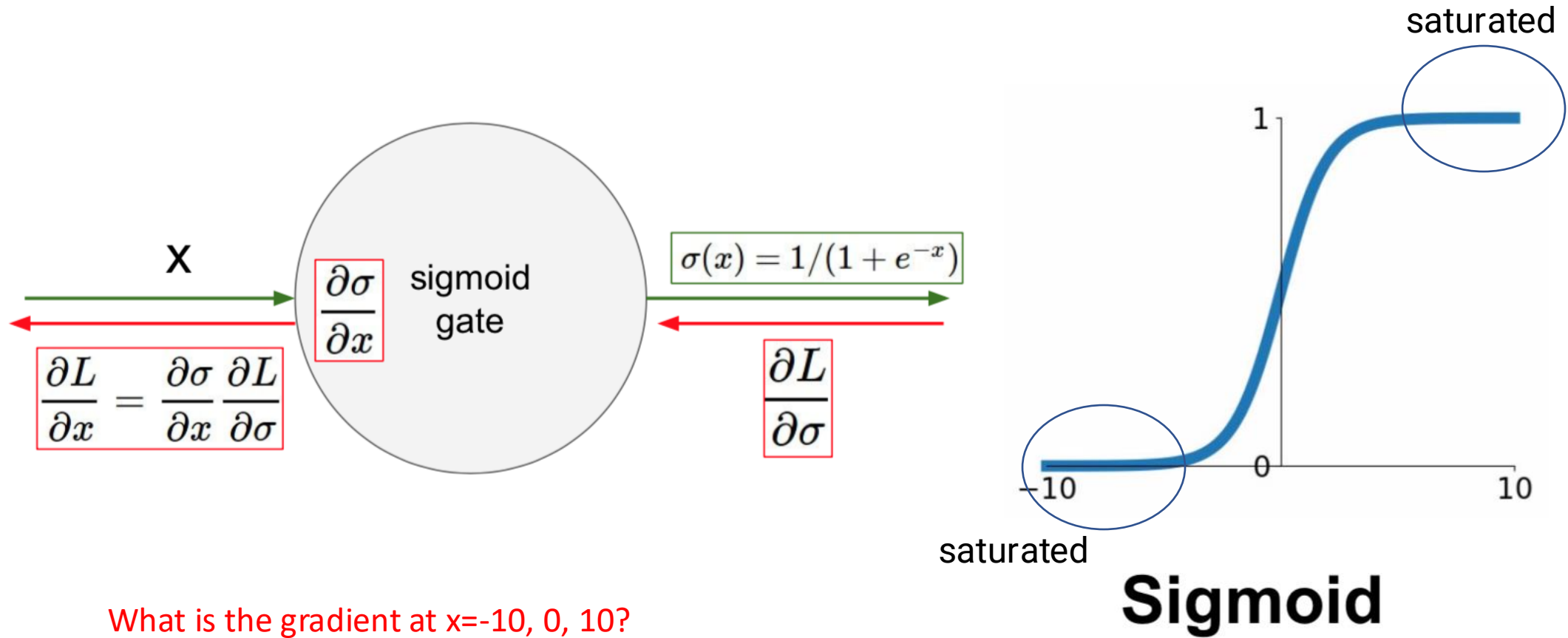
Activation Functions



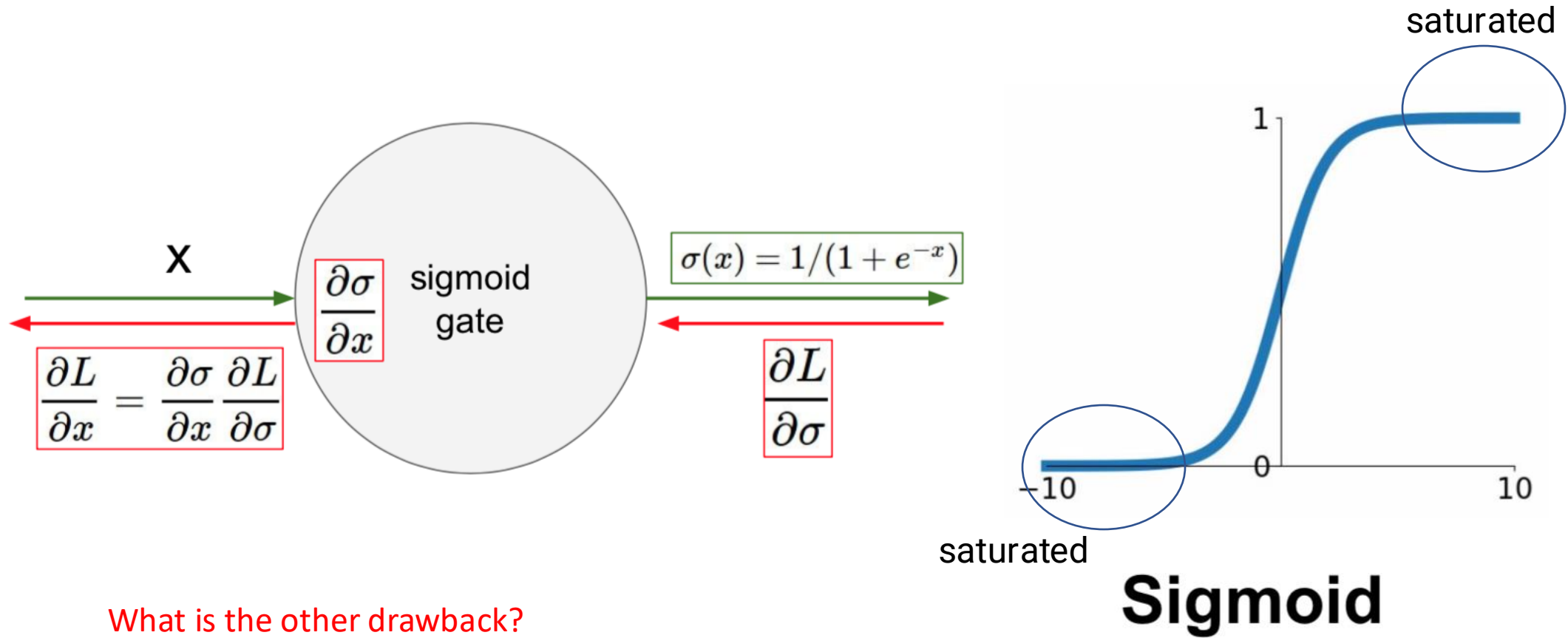
Sigmoid

What is the gradient at $x=-10, 0, 10$?

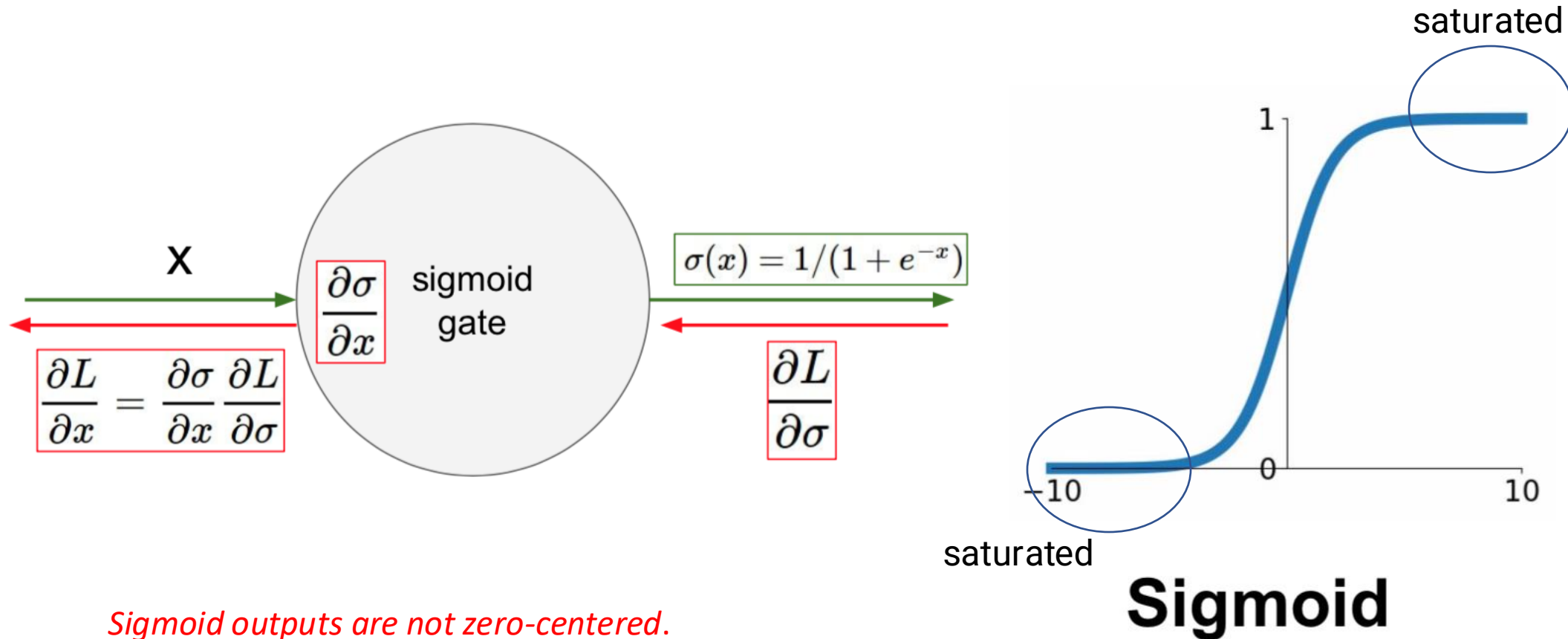
Activation Functions



Activation Functions



Activation Functions



Sigmoid outputs are not zero-centered.

The gradient on the weights w during backpropagation become either all positive, or all negative (depending on the gradient of the whole expression $\frac{\partial L}{\partial \sigma}$), given $f = wTx + b$ when $x > 0$.

(For a single element! Minibatches help)

Activation Functions

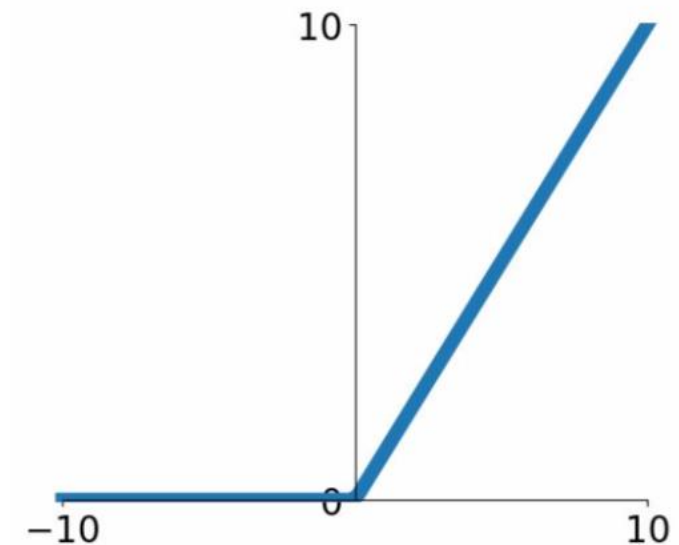
Pros:

- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)

Cons:

- Not zero-centered output
- Could result in dead neurons.

$$f(x) = \max(0, x)$$



ReLU

Activation Functions

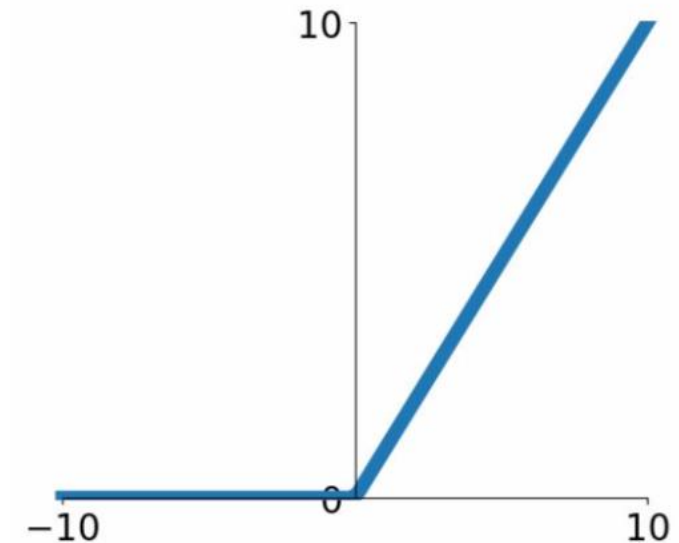
Pros:

- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)

Cons:

- Not zero-centered output
- Could result in dead neurons.

$$f(x) = \max(0, x)$$



ReLU

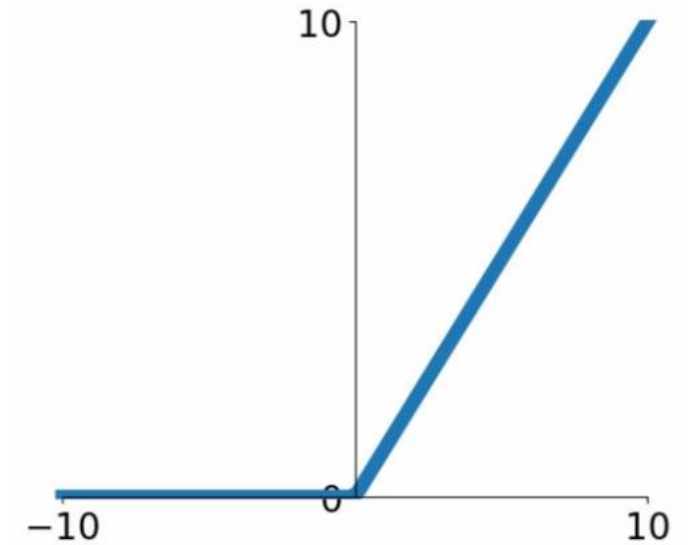
What is the gradient at $x=-10, 0, 10$?

Activation Functions

$$f(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

$$f'(x) = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}$$

$$f(x) = \max(0, x)$$



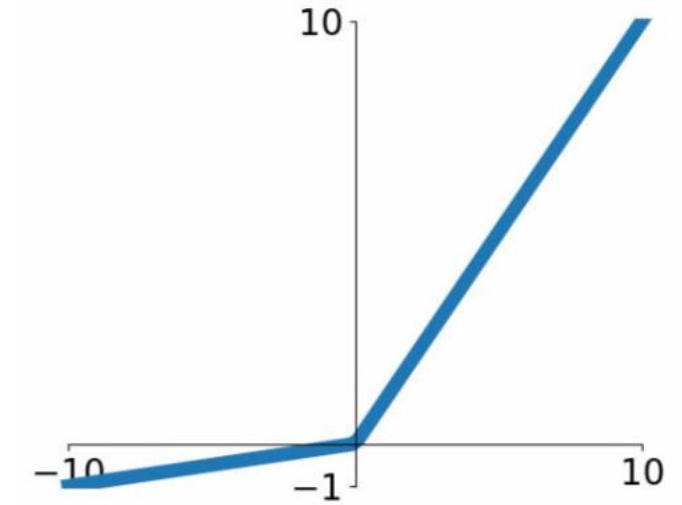
Dying ReLU problem: For activations in the region ($x < 0$) of ReLU, gradient will be 0.

ReLU

Activation Functions

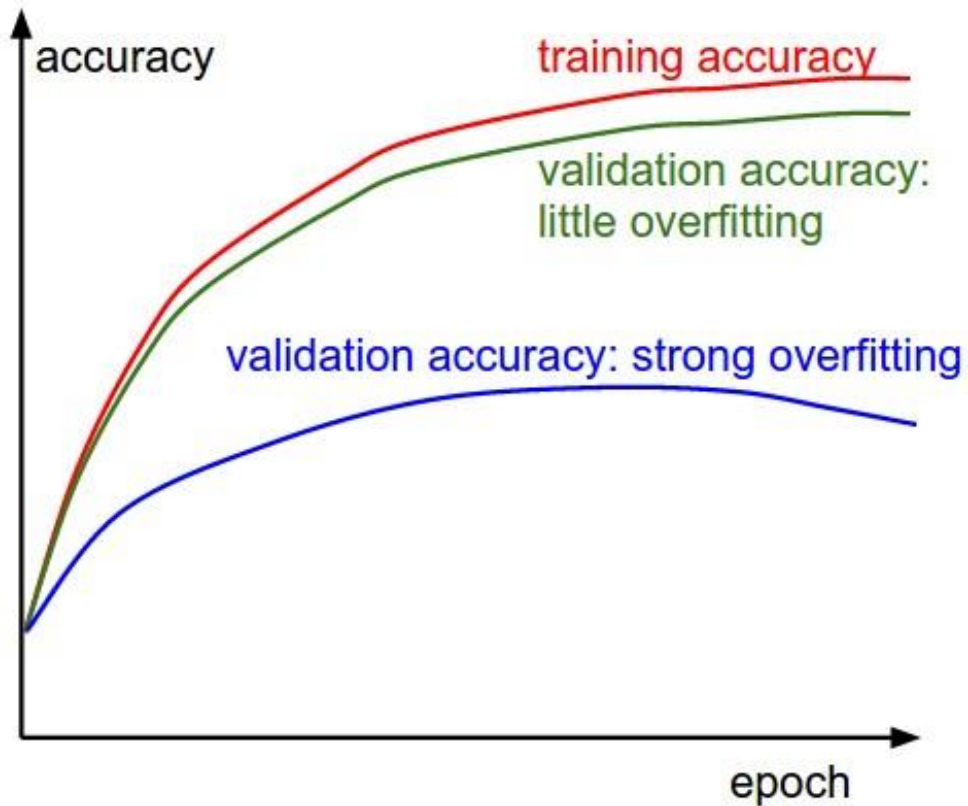
Pros:

- Does not saturate
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)
- Will not die



Leaky ReLU

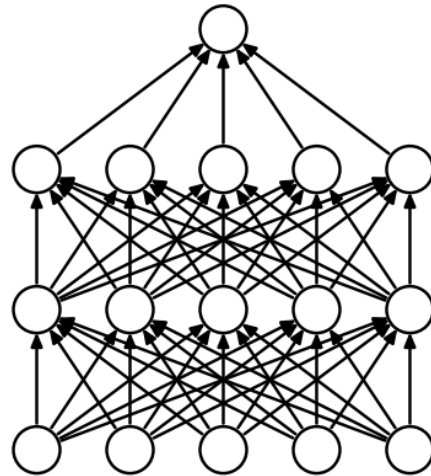
When Overfitting Happens?



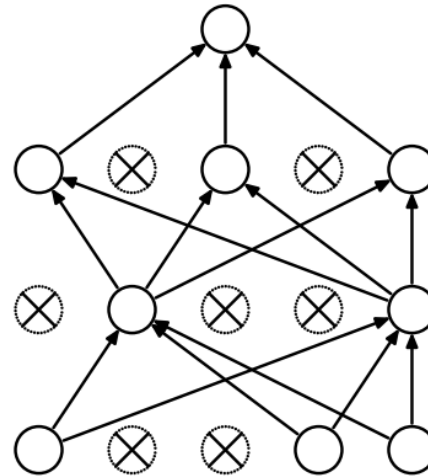
The gap between the training and validation accuracy indicates the amount of overfitting.

Regularizations

- L2 regularization (weight decay): $\frac{1}{2} \lambda w^2$
- L1 regularization: $\lambda |w|$
- L1 + L2: $\lambda_1 |w| + \lambda_2 w^2$
- Max norm: $||w||_2 < c$
- Dropout
- Drop connect



(a) Standard Neural Net



(b) After applying dropout.